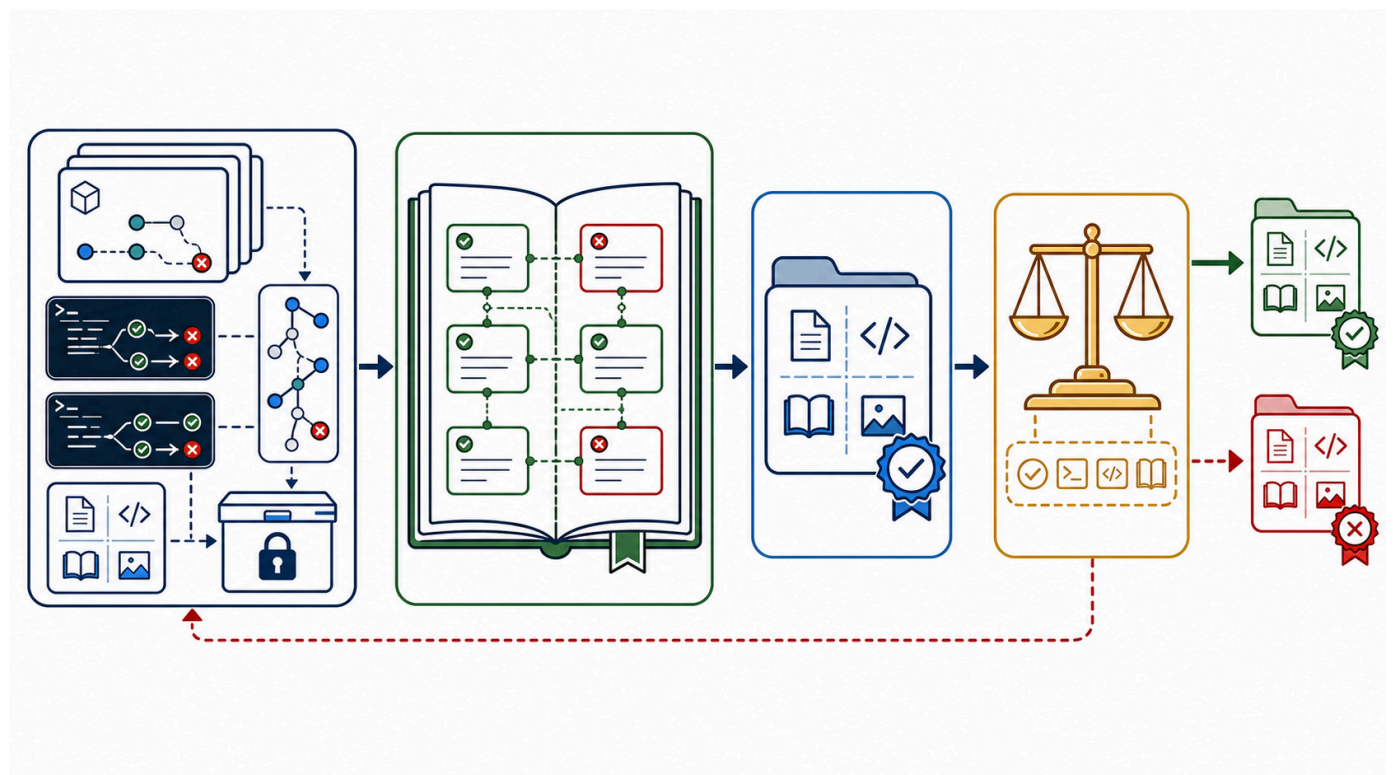


Modern Skill Evaluation and Evolution

A framework selection guide for models, agent harnesses, and reusable skills

Revised final report - 2026-08-30 · Author: Gerardo Villarroel **Primary question:** How can we prove that a versioned skill improved an agent on unseen work without confusing that effect with model, harness, environment, or grader changes?

The modern unit of progress is not a leaderboard score. It is a controlled, reproducible promotion decision backed by task outcomes, trajectories, artifacts, uncertainty, and an untouched holdout.



Conceptual orientation. Immutable traces accumulate into persistent knowledge; knowledge proposes a versioned skill; an independent evaluation gate either promotes the candidate or returns its outcome to the evidence base.

Executive decision

For open and reproducible evaluation of complete skill directories, the strongest ready-made default is:

1. **NVIDIA SkillEvaluator** for skill-native static checks, safety scanning, duplication analysis, and paired live trials that report the incremental **Skill Lift** of a skill.
2. **Harbor** underneath those live trials—or directly when a custom task world is required—for isolated execution, skill provenance, and trial artifacts.
3. **Executable task checks plus calibrated semantic review** for scoring.
4. **Disjoint discovery, development, validation, and holdout cohorts** for controlling adaptive overfitting.
5. **MLflow or Langfuse** when long-lived experiment tracking, production traces, and feedback are required.

6. **An independent promotion gate** that can reject a candidate even when its mean reward, cost, or token count improves.

Use **Inspect AI** instead of Harbor when custom evaluation programs, solver composition, safety controls, rescoring, or heterogeneous sandbox backends matter more than first-class `SKILL.md` handling. Use the **AWS sample skill-eval** as a lightweight CI starter; **Promptfoo** for provider matrices, assertions, and red teaming; **DeepEval** for Python test ergonomics and agent-path metrics; **Ragas** for retrieval-centered systems; and **Pydantic Evals** for typed Python functions and span assertions.

No single product is the complete system. A credible skill-evolution program combines an execution layer, an evidence layer, and a promotion layer.

1. Define the treatment before choosing a framework

Four objects are often called a "harness," but they answer different questions.

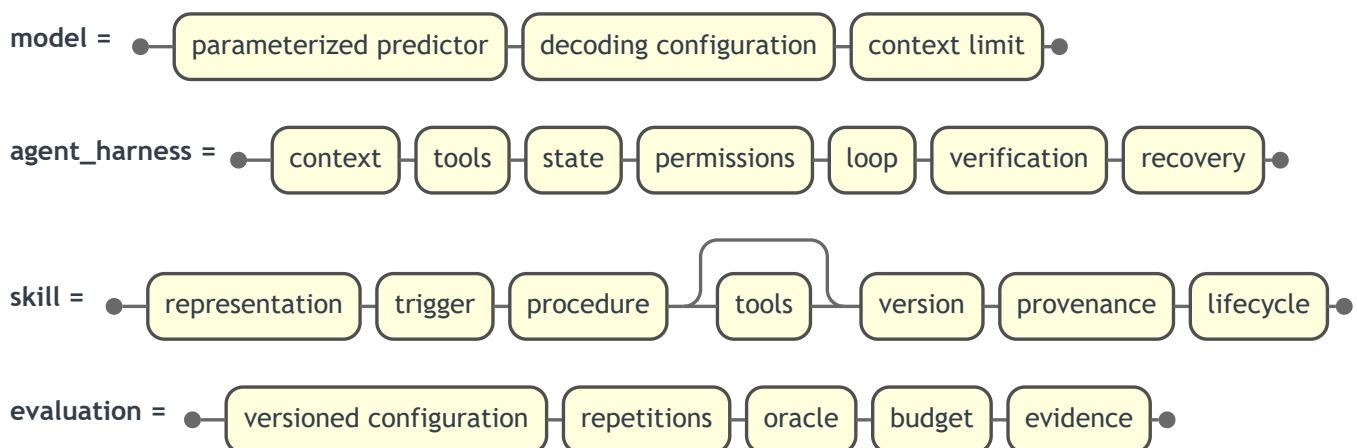


Figure 1. RoadRails, rendered as Mermaid Railroad productions, define each artifact by its operational components. The definitions keep model capability, orchestration, reusable procedure, and measurement policy conceptually separate.

[Animated SVG](#) | [Mermaid source](#)

In a skill evaluation, the **skill is the declared treatment**. The model, agent harness, task version, resource policy, and graders must be frozen or their changes must be modeled explicitly. A run that changes all four objects can describe a product snapshot, but it cannot attribute causality to the skill.

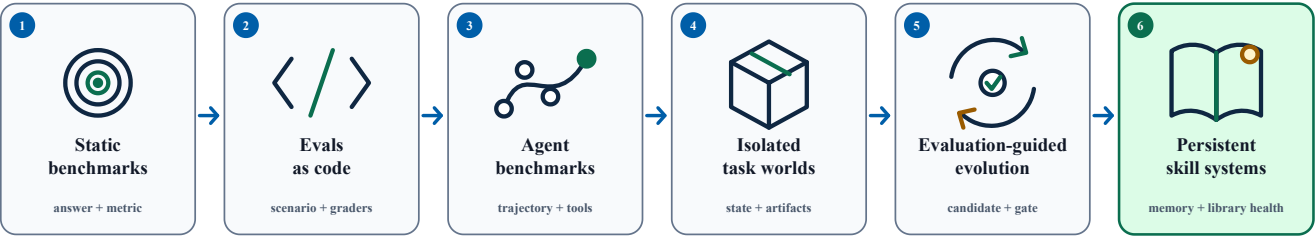
The complete skill directory must receive an immutable identity. Digest only the `SKILL.md` file and an optimizer can silently change a helper script, reference, or asset without changing the recorded treatment.

2. How evaluation reached the skill era

The history is cumulative. Each stage retained earlier controls and added a new unit of observation.

Evaluation evolved by adding evidence—not by discarding it

Each generation changes the unit of observation and keeps the controls that still matter.



What survives into each generation

Amber adds a new evidence component; teal retains it in every later generation.

	Static benchmarks	Evals as code	Agent benchmarks	Isolated task worlds	Evaluation-guided evolution	Persistent skill systems
Answer outcome + metric	+	✓	✓	✓	✓	✓
Versioned scenarios + layered graders	-	+	✓	✓	✓	✓
Trajectory + tool evidence	-	-	+	✓	✓	✓
State + artifacts + resource policy	-	-	-	+	✓	✓
Skill identity + candidate lineage	-	-	-	-	+	✓
Persistent knowledge + library health	-	-	-	-	-	+

+

 added here

✓

 retained

-

 not yet present

Figure 2. Each generation adds a new observable or control while retaining the useful components of earlier systems. Icons identify the new unit; the matrix makes persistence across generations explicit.

D3 authoring source

2.1 Static benchmarks established comparability

A frozen test set and metric made systems comparable, but usually observed only a final answer. Experiment trackers such as MLflow made configurations, runs, datasets, and artifacts durable. That provenance remains foundational.

2.2 Holistic evaluation replaced one score with a profile

HELM standardized scenarios, exposed raw model outputs, and evaluated several qualities rather than treating accuracy as sufficient. The result was a measurement profile: capability alongside robustness, calibration, fairness, efficiency, and other constraints.

2.3 Evals as code made evaluation part of delivery

OpenAI Evals helped normalize versioned datasets, programmable graders, and model-graded checks. These systems are excellent when the natural unit is a response. They do not automatically create a realistic terminal, repository, browser, or other stateful world.

2.4 Agent benchmarks made trajectories observable

AgentBench (paper) evaluated agents in interactive environments. SWE-bench (paper) connected natural-language issues to real repositories and executable tests. The agent now had to inspect state, select tools, modify artifacts, and survive a multi-step loop.

2.5 Isolated task worlds made complete work verifiable

[Terminal-Bench 2.0](#) ([paper](#)) packages realistic terminal tasks with task-specific environments and tests. [Harbor](#) ([repository](#)) generalizes this style of execution into an agent and model evaluation framework. Harbor is documented as a framework rather than by a canonical Harbor-framework paper; similarly named HARBOR papers describe different systems. The environment becomes part of the experimental boundary, not background plumbing.

This distinction is measurable. Anthropic reported a six-percentage-point spread between its least- and most-resourced Terminal-Bench 2.0 configurations, with $p < 0.01$. Small leaderboard gaps can therefore be infrastructure effects rather than agent improvements. See [Quantifying infrastructure noise in agentic coding evals](#).

2.6 Trace-guided optimizers turned failures into candidates

[DSPy](#) treats LM programs as optimizable artifacts. [GEPA](#) reflects on execution evidence, proposes textual changes, and keeps complementary candidates on a Pareto frontier. [Trace2Skill](#) distills trajectory-local lessons into transferable skill directories.

An optimizer does not reduce the need for evaluation. It increases it. Repeatedly searching against visible feedback creates selection bias, which is why the final promotion evidence must come from a sealed cohort that the optimizer did not inspect.

2.7 Skill-native evaluation isolated the value of the skill

The [ACES](#) [paper](#) and its open-source [NVIDIA SkillEvaluator](#) implementation make the skill itself the first-class evaluation target. Deterministic structure and safety checks are followed by paired live trials with and without the skill under the same task, model, harness, workspace, and scorer. The resulting **Skill Lift** estimates the skill's incremental contribution instead of reporting only total agent capability.

This paired design is the decisive methodological advance for skill evaluation. A good agent score does not prove that the skill helped; only a controlled no-skill/with-skill comparison can make that attribution.

2.8 Persistent knowledge turned discarded trials into cumulative evidence

[WikiSkill](#), a Google Research and Virginia Tech preprint submitted on 2026-08-27, adds a durable learning layer between execution traces and executable skills. Its workspace separates an immutable **Raw Layer**, a persistent **Wiki Layer**, and a gated **Skills Layer**. A Wiki Maintainer consolidates failure patterns, successful strategies, proposal history, and validation outcomes. A Skill Proposer uses that accumulated knowledge plus recent traces to produce candidates. Validation can roll a skill back, but it does not roll the wiki back.

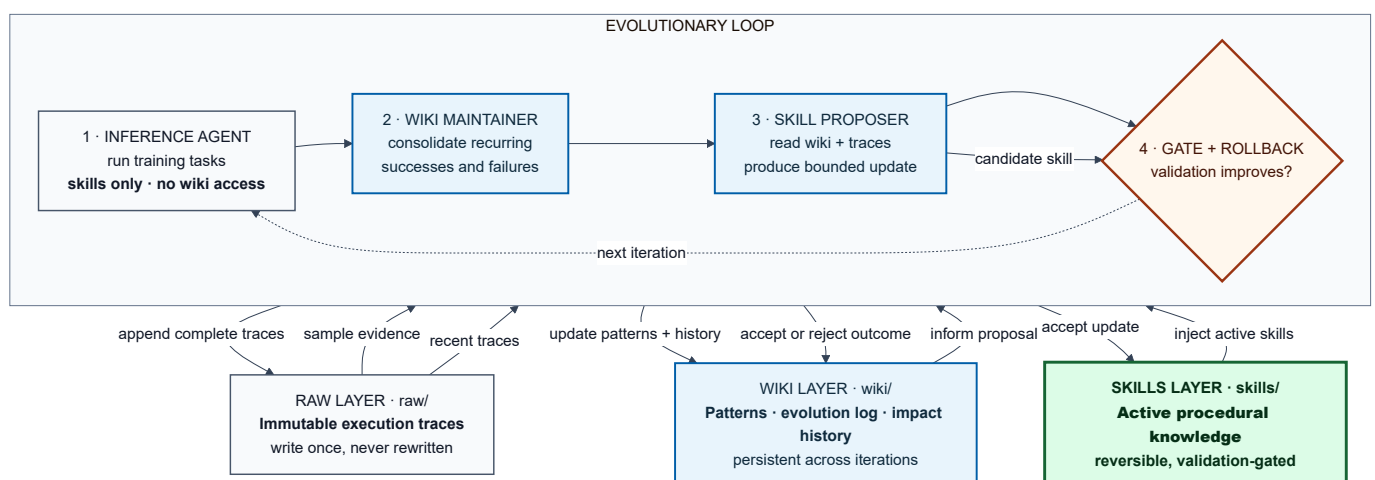


Figure 3. Original reconstruction of the control flow in WikiSkill Figure 2. Immutable traces feed a persistent wiki; the proposer converts accumulated knowledge into gated skills; and the inference agent receives active skills but never the wiki itself. [Animated SVG](#) | [Mermaid source](#)

The separation is empirically consequential within the paper's protocol. In a four-benchmark Gemini-3.5-Flash ablation, giving the Skill Proposer wiki access while withholding it from the Inference Agent raised the reported average from **48.7% to 63.7%**. Giving the Inference Agent wiki access during training reduced that result to **60.9%**. The paper also reports that evolved skills can transfer across models and sometimes outperform self-evolved skills. This distinguishes the ability to **discover procedural knowledge** from the ability to **execute it**.

WikiSkill should currently be treated as a research design, not a drop-in open-source evaluation framework. The study directly injects active skills, so it does not evaluate skill retrieval or triggering; validation accepts only immediately improving proposals; the wiki has no automated pruning mechanism; and the arXiv record does not link a public implementation or code license as of 2026-08-30.

3. The architecture of a modern skill evaluation

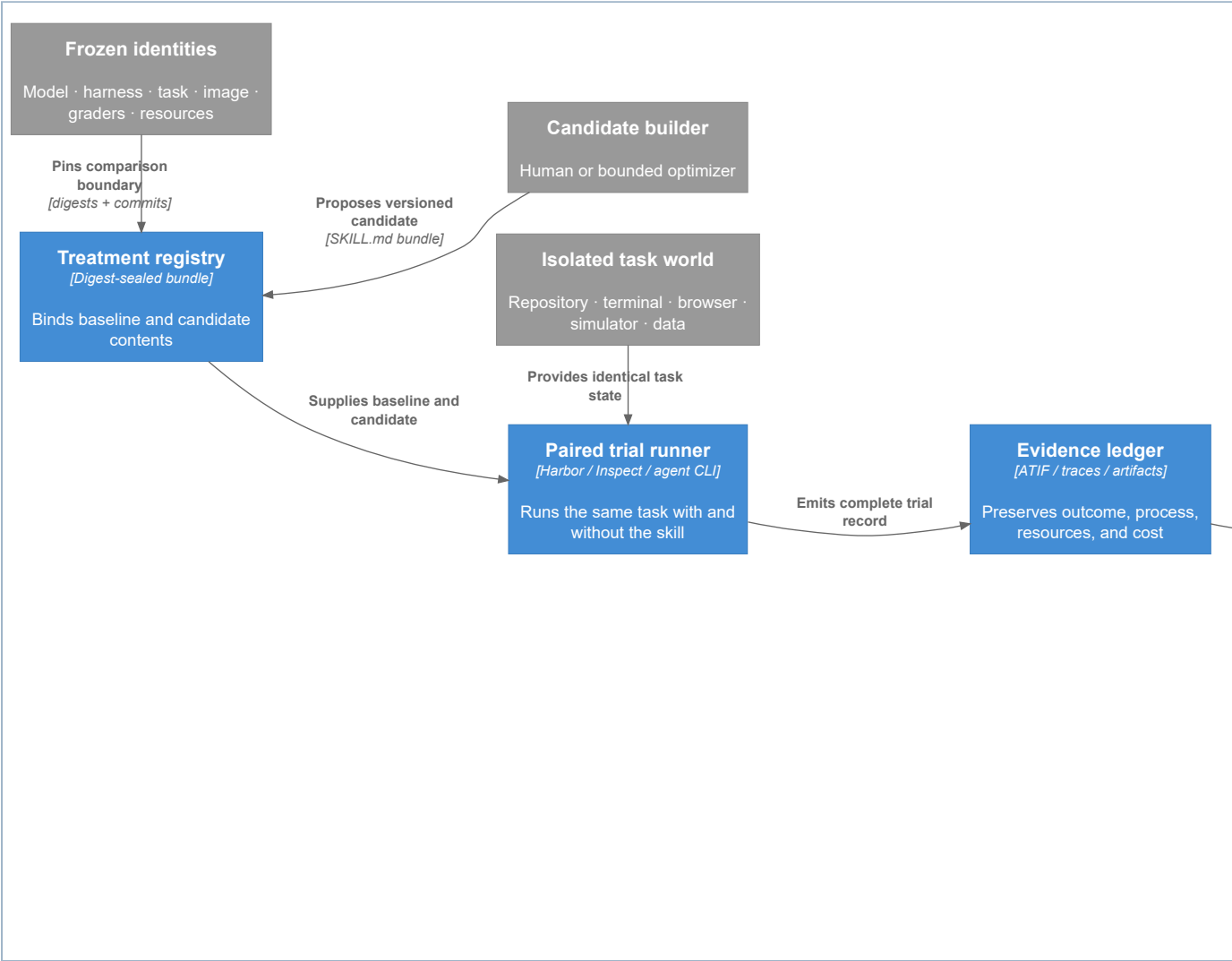


Figure 4a. Controlled treatment and paired execution.

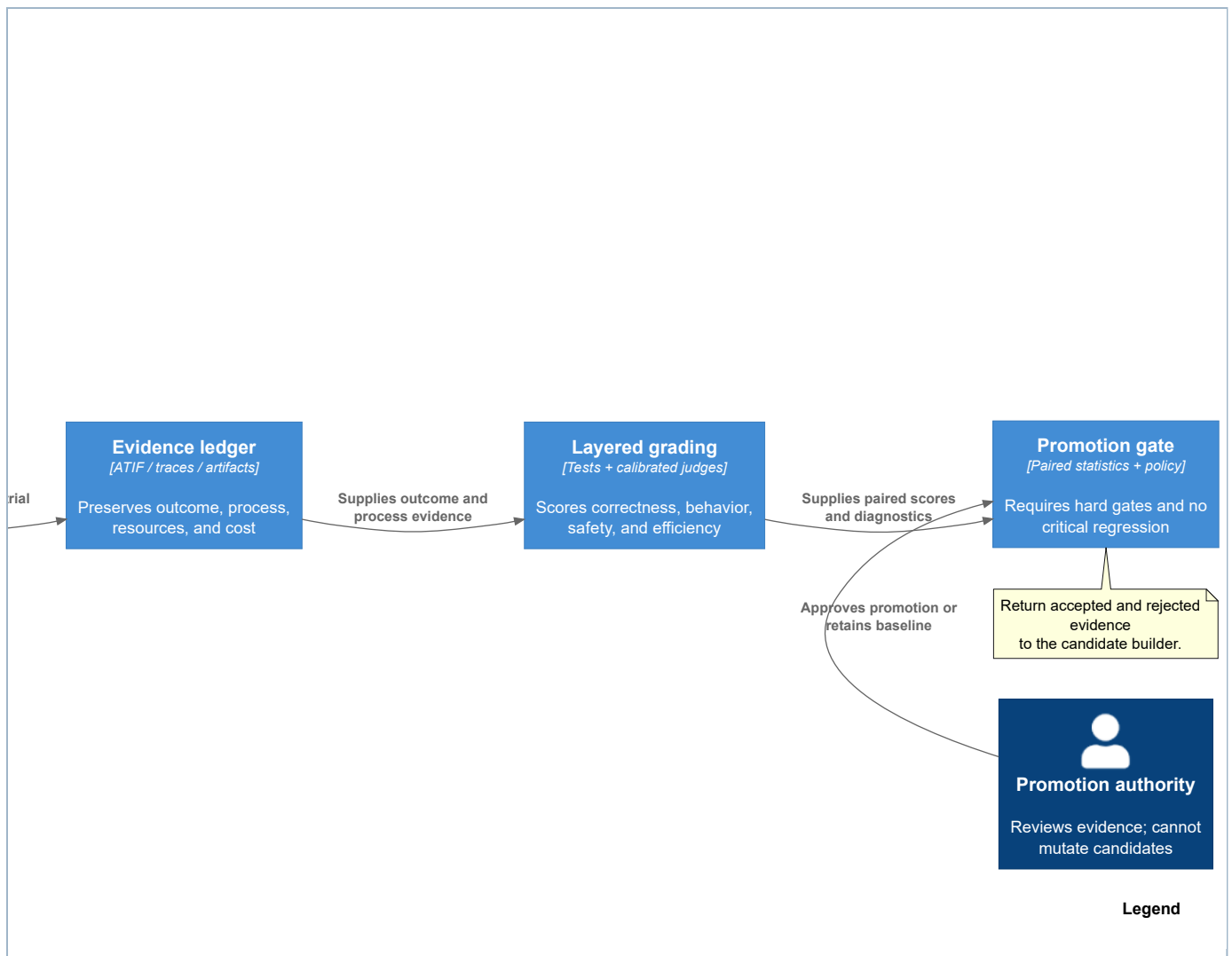


Figure 4b. Evidence, grading, and independent promotion.

Figure 4. Horizontal C4 container view. A study owner freezes the experiment; the orchestrator runs paired baseline and candidate trials in an isolated task world; evidence and scoring remain separate from the independent promotion authority.

[PlantUML C4 source](#)

A credible system has ten pillars.

1. **One declared treatment.** Vary the skill while freezing the model, agent harness, task version, environment, resources, and graders.
2. **Realistic isolated tasks.** Evaluate the work the skill is intended to improve. Repository skills need repositories and tests; browser skills need browser state; RAG skills need a controlled evidence corpus.
3. **Immutable identity.** Digest the complete skill bundle, task, image, verifier, dataset manifest, model settings, and harness commit.
4. **Layered scoring.** Prefer deterministic checks for observable outcomes. Add model or human judgment only where correctness cannot be encoded completely.
5. **Disjoint data roles.** Separate discovery, development, validation, and holdout. Once inspected, a holdout becomes development data for the next generation.
6. **Trajectory evidence.** Preserve messages, tool calls, state transitions, produced artifacts, verifier output, time, tokens, and cost.
7. **Hard gates plus tradeoffs.** Correctness, safety, and semantic quality are gates. Cost, latency, and token use can be Pareto objectives.

8. **Uncertainty and pairing.** Pair baseline and candidate on the same tasks and seeds when possible. Use repeated runs or paired resampling when stochasticity matters.
9. **Failure taxonomy.** Separate semantic failure from provider, infrastructure, evaluator, authentication, and environment failure. Never retry meaning until it passes and then count only the success.
10. **Independent promotion.** Keep the mutator, executor, evaluator, selector, and release authority conceptually separate.

The [Reusable Holdout](#) formalizes the broader problem: adaptive reuse of evaluation data can overfit the evaluation itself. Agent and skill optimizers make this risk operational rather than theoretical.

WikiSkill sharpens the evidence pillar: immutable traces, accumulated learning memory, and the executable skill should have independent identities. Only the skill is the test-time treatment. The wiki is optimizer state whose contents and access policy must be recorded so that a gain is not misattributed to hidden execution context.

4. Framework capability landscape

Capability landscape: where each framework is strongest

Documented support as of 2026-08-30. This is a capability map, not a benchmark score.

	Skill identity	Paired skill lift	Stateful world	Trace evidence	Executable checks	Behavior + semantics	Safety + static	Search + evolution	Default fit
SKILL-SPECIFIC EVALUATORS									
NVIDIA SkillEvaluator	N	N	N	N	N	N	N	A	Three-tier skill QA and live lift
AWS sample skill-eval	N	N	A	S	S	N	N	-	Lightweight audit and CI starter
SkillTester	N	N	A	S	S	S	N	-	Paired utility and security QA
TASK RUNNERS									
Harbor	N	A	N	N	N	S	A	N	Complete skill directories
Inspect AI	A	A	N	N	N	N	S	A	Custom agent research
EVALUATION LIBRARIES									
Promptfoo	A	A	A	S	N	N	N	S	Provider matrices and CI
OpenAI Evals	A	A	-	A	S	N	A	-	Response datasets and graders
DeepEval	A	A	-	N	S	N	S	S	Python tests and agent paths
Ragas	A	A	-	S	S	N	A	A	Retrieval and RAG quality
Pydantic Evals	A	A	-	S	N	S	A	-	Typed Python applications
EVOLUTION AND LIBRARY MAINTENANCE									
Microsoft SkillOpt	N	S	A	N	A	S	A	N	Validation-gated skill training
SkillOps	N	-	A	S	N	A	N	N	Typed library health and repair
LIFECYCLE PLATFORMS									
MLflow	A	A	-	N	N	N	A	A	Experiment lifecycle
Langfuse	A	A	-	N	S	N	A	A	Open-core LLM observability
Phoenix	A	A	-	N	S	N	A	A	Tracing and evaluation
LangSmith	A	A	A	N	S	N	A	A	Managed LangChain lifecycle

N Native / first-class
 S Strong documented support
 A Adapter or manual convention
 - Outside the framework's center of gravity

Read by column, not by row total: complete skill evolution requires execution, evidence, and promotion controls that often span multiple products.

Figure 5. D3-generated ordinal capability map. N means native or first-class, S means strong documented support, A means an adapter or manual convention, and - means outside the framework's center of gravity. Dedicated skill evaluators are separated from task runners, libraries, evolution methods, and lifecycle platforms. The figure is not a quality score and row totals are intentionally meaningless. [D3 authoring source](#)

4.1 Dedicated skill evaluators

Framework	Native center of gravity	Skill-evaluation advantage	Critical caveat
NVIDIA SkillEvaluator	Tier 1 deterministic structure, safety, and script checks; Tier 2 similarity and deduplication; Tier 3 live paired trials through Harbor	Treats directories as first-class artifacts and compares no-skill with with-skill runs under a fixed protocol; reports triggering, behavior, cost, and Skill Lift	Young project; live trials still require a provider, agent runtime, task workspace, sandbox, and reviewed task generation
AWS sample skill-eval	Lightweight safety, quality, reliability, and cost checks	Small MIT-0 starter that is easy to place in authoring or CI workflows	Sample implementation rather than a general stateful task runner or mature benchmark ecosystem
SkillTester	Paired utility and security probes for skills	Makes baseline-versus-skill comparison and adversarial behavior explicit	Repository has no detected license; source visibility alone does not grant reuse rights

Decision: choose NVIDIA SkillEvaluator when a ready-made, open-source, skill-native quality gate is the goal. Use the AWS sample for a smaller CI starting point. Treat SkillTester as a research/service option until its code has an explicit license.

The independent [framework-at-scale study](#) reinforces the paired design: it evaluated 500 skills, 1,000 generated tasks, and 19 agent-model configurations, finding that adherence and gains vary materially by runtime. A skill is therefore not universally “good”; its effect is conditional on the model and harness in which it executes.

4.2 Task runners

Framework	Native center of gravity	Skill-evaluation advantage	Critical caveat
Harbor	Containerized agent/model trials, task verifiers, artifacts, and optimization	A local or Git skill directory is a first-class input; source, digest, and resolved commit can be locked	Provenance does not create correct dataset splits or an independent promotion policy
Inspect AI	Python evaluation programs, solvers, scorers, logs, limits, and multiple sandbox backends	Strong control over custom agents, safety studies, rescoring, and heterogeneous execution	A skill bundle is an experiment convention or adapter rather than the native unit

Decision: choose Harbor when the object being evolved is a complete skill directory and tasks change a stateful world. Choose Inspect AI when evaluation-program flexibility, safety controls, or sandbox diversity is the primary requirement.

4.3 Evaluation libraries

Framework	Best fit	Evidence and scoring	Skill-evolution limit
Promptfoo	Prompt/provider matrices, CI assertions, and red teaming	Declarative assertions, custom code, model graders, latency, and cost	Provider runtime is not a general isolated task world
OpenAI Evals	Response datasets and custom graders	Exact, custom, and model-graded evaluation	No general skill bundle or agent sandbox in the OSS runner
DeepEval	Python and pytest workflows, including agent-path evaluation	Task metrics, tool/sub-agent behavior, custom DAGs, and model judges	Prompt optimization is not a generic population search over complete skill directories
Ragas	Retrieval, RAG, and context quality	Retrieval/generation metrics plus message and tool evaluation	Pair with a task runner when the agent changes external state
Pydantic Evals	Typed Python applications	Typed evaluators and OpenTelemetry span assertions	Function runtime is not an isolated task-world abstraction

4.4 Lifecycle and observability platforms

Framework	Best fit	What it adds	What it does not replace
MLflow	Broad experiment and model lifecycle	Runs, datasets, artifacts, traces, scorers, registry, and production feedback	A clean terminal or browser task world
Langfuse	Self-hosted or managed LLM observability	Traces, datasets, experiments, scores, and user feedback	First-class skill locking and holdout governance
Phoenix	OpenInference/OpenTelemetry tracing and evaluation	Spans, datasets, code/model/human evaluation	OSI-open licensing and isolated task execution
LangSmith	Managed LangChain-centered evaluation	Datasets, trajectories, pairwise evaluators, online traces, and feedback	An open-source, portable task runner

These platforms complement a dedicated evaluator and a task runner. They answer how experiments and production behavior are stored, compared, and monitored; they do not automatically prove that a candidate skill caused a result.

4.5 License and community snapshot

GitHub stars and forks are rough adoption signals, not evidence of evaluation validity. Counts below are a dated snapshot from the GitHub API on **2026-08-30**.

Framework	License posture	Stars	Forks	Operational implication
NVIDIA SkillEvaluator	Apache-2.0	354	34	Open, dedicated skill-evaluation implementation
AWS sample skill-eval	MIT-0	14	3	Small, permissive reference implementation
SkillTester	No detected repository license	36	2	Source-visible; seek permission before reuse
Harbor	Apache-2.0	4,774	1,683	Open task runner and optimization substrate
Inspect AI	MIT	2,663	682	Open Python research and evaluation runtime
Promptfoo	MIT	24,672	2,250	Large JS/TS and CI-oriented community
OpenAI Evals repository	MIT code; individual datasets retain their own terms	19,307	3,069	OSS runner is distinct from proprietary hosted services
DeepEval	Apache-2.0	17,962	1,875	Python/pytest evaluation ecosystem
Ragas	Apache-2.0	15,544	1,662	Retrieval and RAG-centered ecosystem
Pydantic AI repository	MIT	19,583	2,616	Count covers the wider repository, including Pydantic Evals
Microsoft SkillOpt	MIT	16,484	1,549	Large early community around skill optimization
SkillOps	MIT	62	6	Early project for typed skill-library operations
MLflow	Apache-2.0	27,735	6,234	Broad lifecycle and observability platform
Langfuse	MIT core; enterprise directories have separate terms	33,923	3,666	Open core with self-hosting and managed options

Framework	License posture	Stars	Forks	Operational implication
Phoenix	Elastic License 2.0	11,247	1,084	Source available, not OSI open source; hosted-service restrictions apply
LangSmith	Proprietary	Not comparable	Not comparable	Managed service and LangChain integration

“Public on GitHub” is not a license. Without an explicit license, default copyright applies. Review the [Open Source Definition](#) and the exact license file before adopting, modifying, or redistributing any framework.

4.6 Evolution and library operations are not evaluation frameworks

Method	Distinctive evolution state	Evaluation it still requires	Adoption posture
Microsoft SkillOpt	Bounded add/delete/replace edits, rejected-edit memory, validation gates, and slow meta-updates	Real task execution, independent splits, calibrated scoring, and an untouched promotion holdout	MIT implementation and paper from Microsoft Research
SkillOps	Typed skill contracts plus graph health, merge, repair, retirement, validators, and adapters for skill libraries	Runtime utility, triggering, behavior, safety, and generalization evidence	Separate MIT project from Emory/UIUC; paper
GEPA	Reflective textual proposals plus a Pareto frontier	A task runner, case-level feedback, disjoint validation, and a sealed holdout	Available through open DSPy tooling; integrate with the task world that matches the skill
Trace2Skill	Trajectory-local lessons consolidated into transferable skill directories	Independent execution and promotion evidence beyond the traces used to distill	Research method; verify the implementation and license selected for use
WikiSkill	Immutable raw traces, a persistent wiki, proposal impact history, and gated skills	Real task worlds, calibrated graders, holdout governance, and skill-retrieval evaluation	Public preprint; no implementation or code license is linked from the arXiv record as of 2026-08-30

Naming clarification: **SkillOpt** is Microsoft’s optimizer. **SkillOps** is the separate Emory/UIUC skill-library project. Microsoft also has an unrelated ACES cyber-evaluation repository; the **ACES skill-evaluation method** discussed here is the paper implemented by NVIDIA SkillEvaluator.

The distinction matters: an optimizer proposes or selects treatments; it does not prove that they generalize. SkillOpt, SkillOps, GEPA, Trace2Skill, and WikiSkill can sit above Harbor or Inspect AI, with MLflow or Langfuse preserving lifecycle evidence, but the evaluator and promotion authority must remain independently specified.

5. Select from the artifact that must be correct

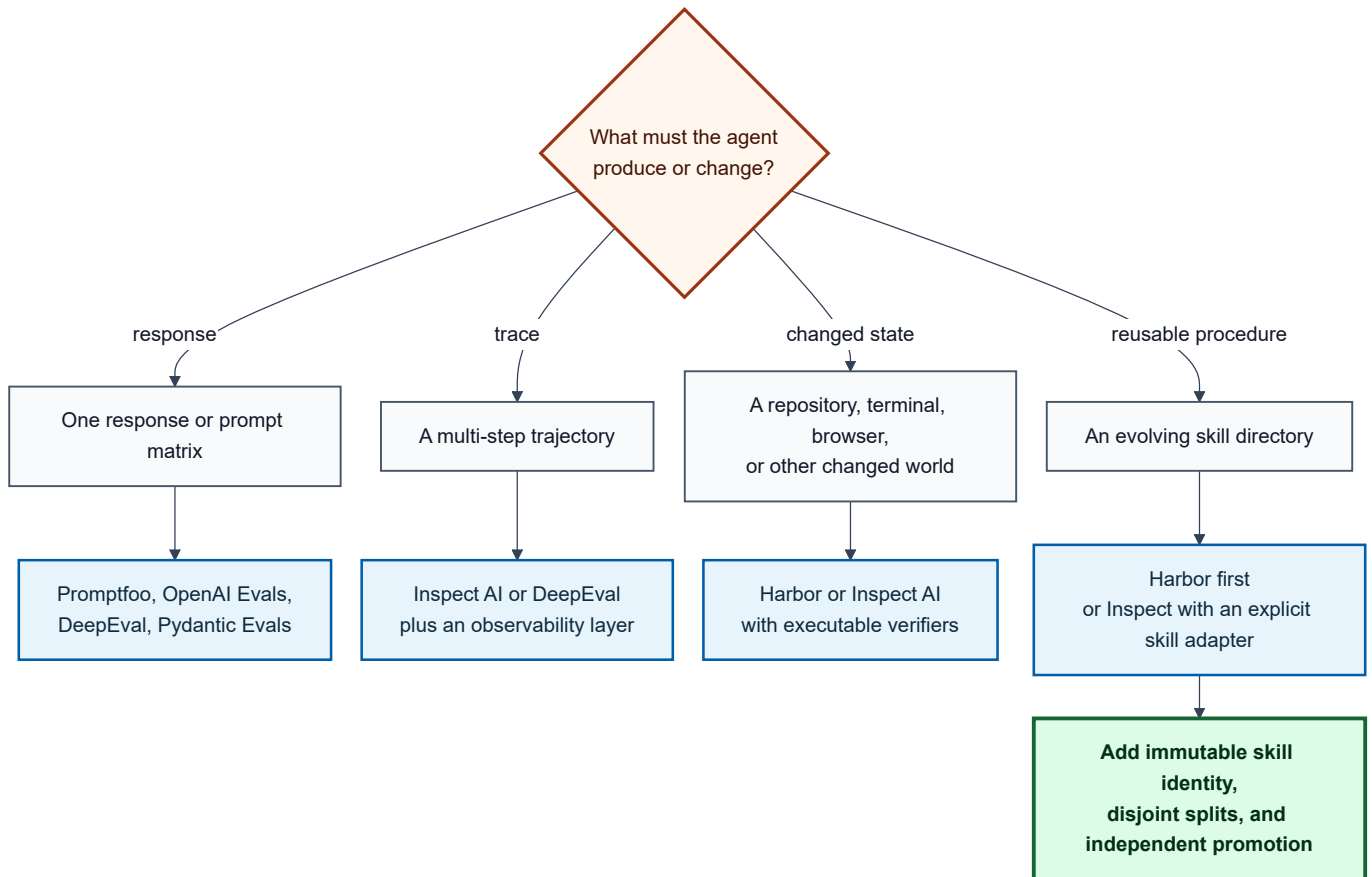


Figure 6. Start with the artifact that must be correct, not with a vendor feature list. An evolving skill requires response, trajectory, and changed-world evidence.

[Animated SVG](#) | [Mermaid source](#)

Use these six factors to select or compose the stack.

5.1 Treatment boundary

- Can the framework identify the complete skill directory by digest and source commit?
- Can it keep the model and harness constant while injecting baseline and candidate?
- Can it preserve parent-child lineage across generations?

5.2 World fidelity

- Is the task answer-only, function-level, terminal-based, repository-based, browser-based, or multi-agent?
- Can CPU, RAM, time, network, images, credentials, and external services be pinned?
- Does the final verifier inspect the changed world rather than trusting the agent's final message?

5.3 Grader strength

- Are executable checks available for outcomes that can be observed directly?
- Does semantic review cite the evidence it used?
- Has any model judge been calibrated against blinded human review?
- Can old logs be rescored when the rubric changes?

5.4 Evidence and statistics

- Are trajectories, artifacts, verifier output, errors, time, tokens, and cost retained?
- Are baseline and candidate paired by task and seed?
- Are counts, effect sizes or intervals, and worst-domain cells reported?
- Are missing trials and infrastructure failures visible rather than silently dropped?

5.5 Optimization and governance

- Is the search budget fixed before results are known?
- Are discovery, development, validation, and holdout identities disjoint?
- Can a hard gate reject a lower-cost or higher-mean candidate?
- Is holdout release one-way and promotion independently reviewed?

5.6 Deployment, license, and ecosystem

- Can the system run locally, air-gapped, in CI, or in the required cloud?
- Is the license compatible with internal modification, redistribution, and hosted use?
- Does the community maintain adapters for the models, sandboxes, and telemetry stack already in use?
- What is the exit strategy for datasets, traces, scorers, and candidate bundles?

6. Promotion protocol for evolving a skill

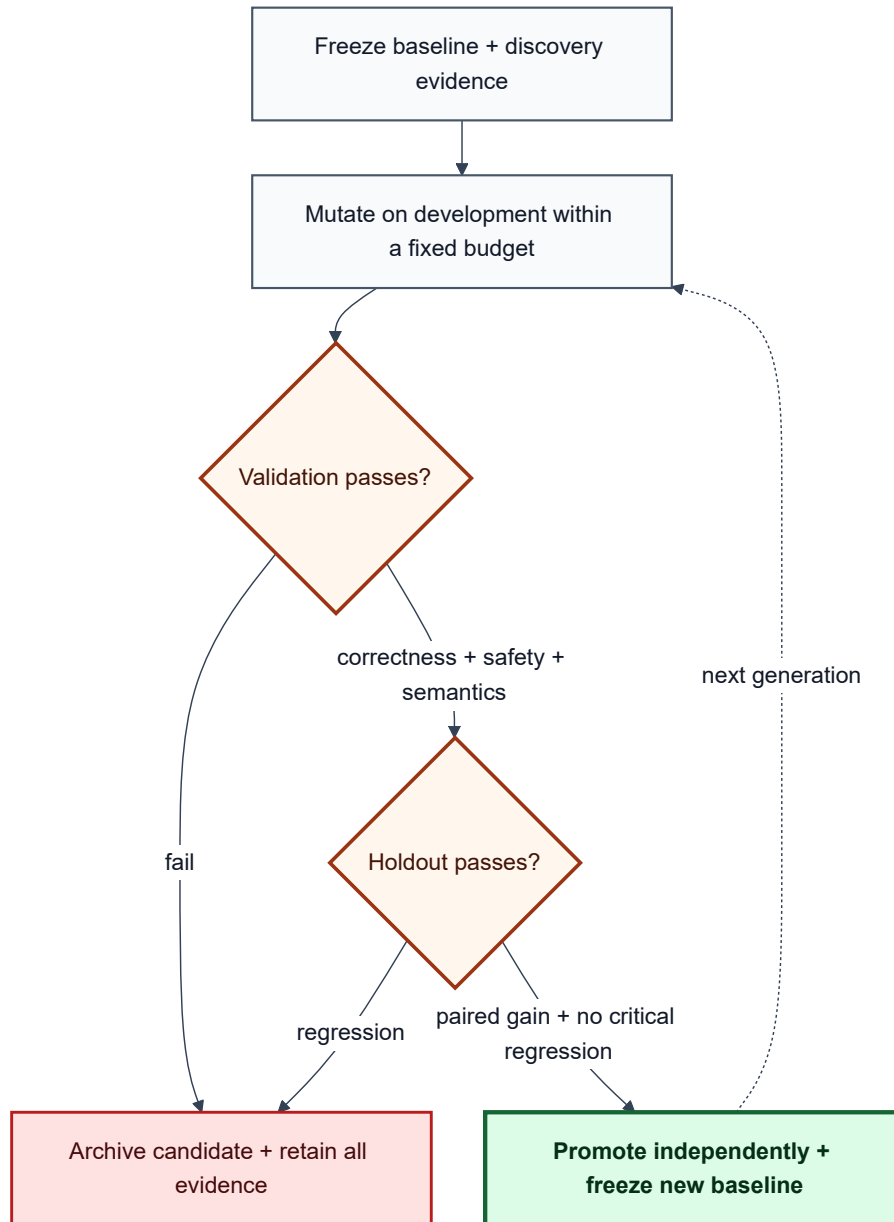


Figure 7. Visible feedback is restricted to development. Validation filters finalists; the sealed holdout decides whether a candidate may replace the baseline.

[Animated SVG](#) | [Mermaid source](#)

Stage 1: Freeze the study

Record task manifests, split membership, model and decoding, harness commit, container digests, resource policy, verifier and rubric digests, baseline skill digest, retry policy, budgets, and promotion rules before evaluating a candidate.

Stage 2: Discover representative failures

Run the frozen baseline. Preserve case-level rewards, trajectories, produced artifacts, verifier diagnostics, and failure classes. Discovery evidence identifies mutation hypotheses; it does not establish generalization.

Stage 3: Search within a fixed budget

Generate candidates with GEPA, trace distillation, Pareto search, operator coevolution, or human review. The optimizer may inspect only the cohorts assigned to development. Keep every candidate's complete bundle and parent lineage.

Stage 4: Validate hard constraints

Use validation to reject unsafe, semantically weak, or domain-regressing candidates. Do not compensate for a critical regression with an unrelated gain in cost or mean reward.

Stage 5: Release the sealed holdout once

Compare the finalist with the frozen baseline on identical unseen tasks. Pair runs when possible, report uncertainty, and show every task cell. After release, retire that holdout from future promotion claims.

Stage 6: Promote independently

Promotion should require all hard gates, no prohibited subgroup regression, acceptable uncertainty, valid lineage, and reviewer approval. The new winner becomes the next frozen baseline; rejected candidates remain useful evidence.

7. Lessons from the local Harbor evolution program

The local work is valuable because it records non-promotions rather than presenting only winners.

The frozen [Skill Arena Harbor comparison](#) contains **24 Harbor jobs and 78 trials** across development and holdout, with no recorded errors or retries. Trace distillation and reflective Pareto search tied for the strongest selected holdout mean. That is evidence for those tasks and budgets, not a universal ranking of optimizers.

The [Knowledge skill-evolution index](#) shows three failure modes that a mean score would miss:

- A candidate won both development datasets and the mean holdout, but regressed one critical holdout cell. A zero-regression rule retained the baseline.
- A candidate passed all four development trials and then failed to qualify on the two-question holdout.
- A token-optimized candidate reduced recorded tokens by **98.674426%** and passed its numeric thresholds, but independent semantic review found regressions in four of six cases. It was rejected.

The conclusion is not that optimization is unreliable. It is that development selects what deserves scrutiny while holdout evidence decides what may be promoted.

Both supporting repositories and the reports cited above are public. No private knowledge-corpus text, retrieval output, or unpublished evaluation artifact is copied into this report.

8. Recommended reference stacks

Workload	Execution and evaluation	Lifecycle evidence	Candidate generation
Dedicated authoring-time and CI quality gate for a skill directory	NVIDIA SkillEvaluator; Harbor for Tier 3 paired execution	Native reports, optionally MLflow or Langfuse	SkillOpt, GEPA, trace distillation, or a bounded custom mutator
Coding, terminal, data transformation, artifact production	NVIDIA SkillEvaluator over Harbor, or direct Harbor for a bespoke evaluator	MLflow or Langfuse when needed	SkillOpt, Harbor GEPA, trace distillation, Pareto search, or a bounded custom mutator
Safety studies, custom loops, multi-agent research	Inspect AI with selected sandbox backend	MLflow, Langfuse, Phoenix, or LangSmith	External optimizer with explicit skill identity

Workload	Execution and evaluation	Lifecycle evidence	Candidate generation
Provider and prompt CI, red teaming	Promptfoo	Native reports or lifecycle platform	Matrix search or external optimizer
Python application and agent tests	DeepEval or Pydantic Evals	OpenTelemetry-compatible platform	Prompt optimizer or external skill mutator
RAG and retrieval systems	Ragas plus Harbor or Inspect when external state changes	MLflow, Langfuse, Phoenix, or LangSmith	Retrieval/prompt optimizer with sealed corpus holdout

For the central use case in this report, the practical default is:

NVIDIA SkillEvaluator for the ready-made skill gate + Harbor for isolated paired execution + deterministic tests and calibrated semantic review for scoring + a sealed holdout and independent reviewer for promotion. Use Harbor directly when the task world or evaluator must be bespoke.

9. Final review checklist

Before calling a candidate skill better, verify all of the following.

- ☐ The complete baseline and candidate skill bundles have different, recorded digests.
- ☐ A paired no-skill/with-skill condition isolates Skill Lift from total agent capability whenever the research question is whether the skill helped.
- ☐ Model, harness, task, environment, resource, verifier, and rubric identities are frozen.
- ☐ Discovery, development, validation, and holdout manifests are disjoint.
- ☐ The optimizer never received holdout prompts, labels, rewards, or traces.
- ☐ Semantic failures were not retried until they passed.
- ☐ Infrastructure recovery retained the original attempt and exact provenance.
- ☐ Raw traces, accumulated knowledge, and the executable skill have separate identities and access policies.
- ☐ Rejected candidate outcomes remain available to the optimizer without entering the test-time context.
- ☐ Every promoted claim includes task-level baseline and candidate results.
- ☐ Uncertainty is reported when sampling or model behavior is stochastic.
- ☐ Worst-domain and safety gates pass; the mean does not conceal a critical regression.
- ☐ Licensing permits the intended modification, redistribution, and deployment.
- ☐ An authority independent from candidate generation approved promotion.

References

- Carnegie Mellon Software Engineering Institute. [Architecture Tradeoff Analysis Method collection](#).
- Agent Skills. [Open specification for reusable agent skills](#).
- Yao et al. [Harness-Bench: Measuring Harness Effects across Models in Realistic Agent Workflows](#).
- Liang et al. [Holistic Evaluation of Language Models](#).
- OpenAI. [OpenAI Evals repository](#) and [Evals API reference](#).
- Liu et al. [AgentBench: Evaluating LLMs as Agents](#).
- Jimenez et al. [SWE-bench: Can Language Models Resolve Real-World GitHub Issues?](#).
- The Terminal-Bench team. [Terminal-Bench: Benchmarking Agents on Hard, Realistic Tasks in Command Line Interfaces](#).

- Anthropic. [Quantifying infrastructure noise in agentic coding evals](#).
- Harbor. [Documentation](#), [skill configuration](#), and [repository](#).
- Kevin et al. [Evaluating Skills, Not Just Agents: Agentic Continuous Evaluation of Skills](#) and [NVIDIA SkillEvaluator](#).
- Shaposhnikov et al. [A Framework for Evaluating Agentic Skills at Scale](#).
- AWS Samples. [Agent skill evaluation sample](#).
- SkillTester. [Paper](#) and [repository](#).
- UK AI Security Institute. [Inspect AI documentation](#) and [repository](#).
- Promptfoo. [Repository](#).
- DeepEval. [Repository](#).
- Ragas. [Repository](#).
- Pydantic. [Pydantic Evals documentation](#).
- MLflow. [Repository](#).
- Langfuse. [Repository](#) and [license](#).
- Arize AI. [Phoenix repository](#) and [Elastic License 2.0](#).
- LangChain. [LangSmith evaluation documentation](#).
- Khattab et al. [DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines](#).
- Agrawal et al. [GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning](#).
- Qian et al. [Trace2Skill: Distill Trajectory-Local Lessons into Transferable Agent Skills](#).
- Yang et al. [SkillOpt: Executive Strategy for Self-Evolving Agent Skills](#), [Microsoft Research overview](#), and [repository](#).
- Song et al. [SkillOps: Towards Automated Skill Library Management](#) and [repository](#).
- Tang et al. [WikiSkill: Compiling Agent Experience into Persistent Knowledge for Skill Evolution](#).
- Dwork et al. [The Reusable Holdout: Preserving Validity in Adaptive Data Analysis](#).
- Skill Arena. [Harbor skill evolution repository](#) and [frozen comparison report](#).
- Knowledge. [Skill exploration and evolution index](#).
- Open Source Initiative. [The Open Source Definition](#).

Bottom line: skill evolution is experimental software engineering. The optimizer may be creative. The evaluator must be versioned, skeptical, independently governed, and difficult to game.